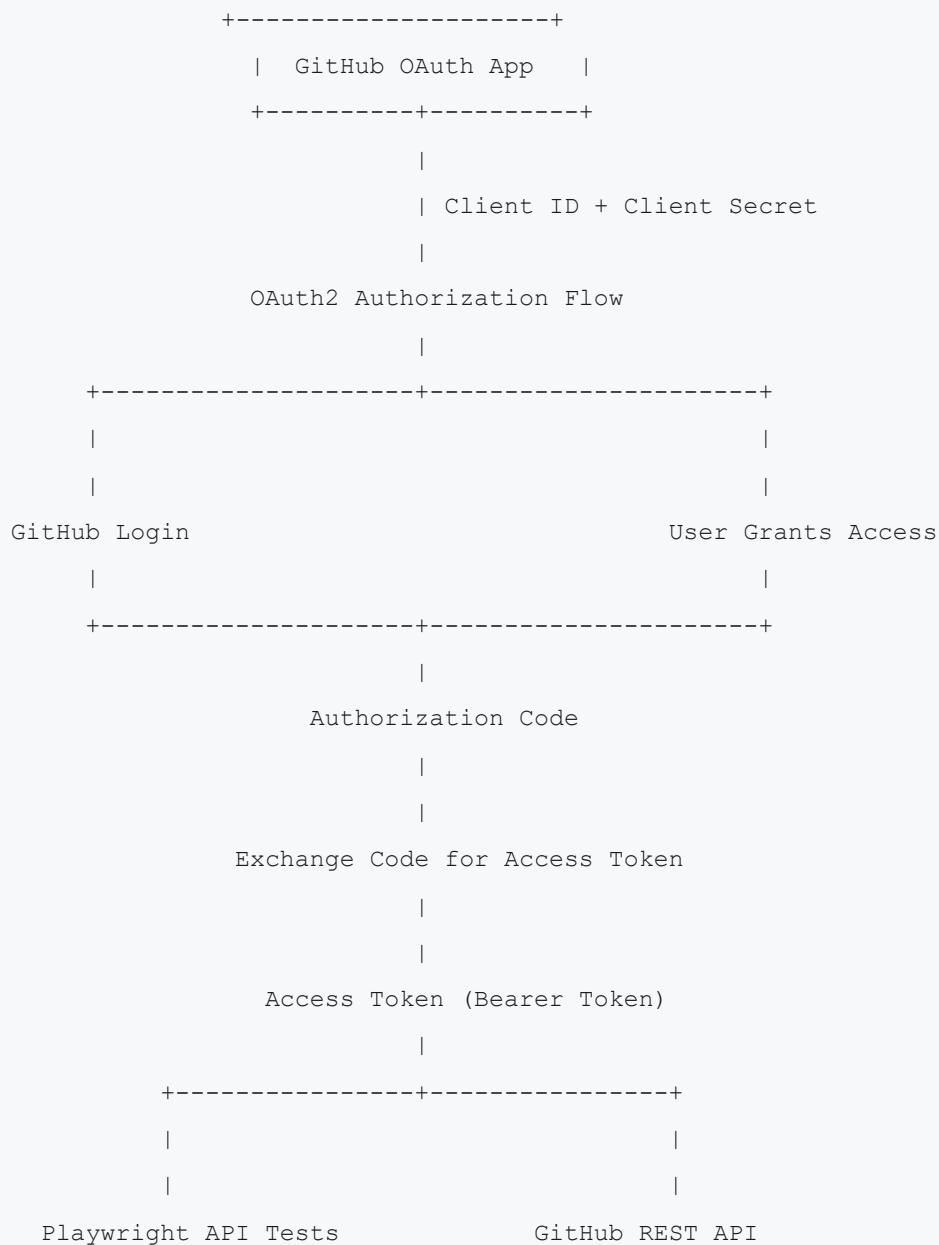


OAuth2 - Authentication

Architecture



What you are going to learn

By the end of this tutorial you'll know how to

- Create a GitHub OAuth App
- Generate an OAuth2 Access Token

- Use the token in Playwright
- Call GitHub REST APIs
- Validate API responses
- Perform CRUD operations
- Handle authentication failures

Step 1: Create GitHub Account

Create a GitHub account if you don't already have one.

Step 2: Create OAuth App

Open GitHub

```
Settings
  ↓
Developer Settings
  ↓
OAuth Apps
  ↓
New OAuth App
```

Fill the details.

```
Application Name
  Playwright OAuth Demo

Homepage URL
  http://localhost:3000

Authorization callback URL
  http://localhost:3000/callback
```

Click

```
Register Application
```

You will receive

```
Client ID
```

Then generate

```
Client Secret
```

Example

```
Client ID
```

```
Iv1.a1234567890abcd
```

```
Client Secret
```

```
abcd1234567890987654321
```

Keep these safe.

Step 3: Understand OAuth2 Flow

GitHub uses the Authorization Code Flow.

```
Playwright
  |
  | Open Browser
  |
GitHub Login Page
  |
User Login
  |
User Grants Permission
  |
Authorization Code
  |
POST
https://github.com/login/oauth/access_token
  |
Access Token
  |
GitHub REST APIs
```

Step 4: Request Authorization Code

Open

```
https://github.com/login/oauth/authorize
```

Parameters

```
client_id

scope

redirect_uri
```

Example

```
https://github.com/login/oauth/authorize
?client_id=YOUR_CLIENT_ID
&scope=repo,user,delete_repo
&redirect_uri=http://localhost:3000/callback
```

GitHub redirects to

```
http://localhost:3000/callback?code=123456
```

The value

```
code
```

is your authorization code.

Important Note

GitHub **Authorization Codes are single-use.**

If you already used:

```
GITHUB_AUTHORIZATION_CODE=2bfe74cdedabcbd9bfcf
```

then GitHub will return:

```
{
  "error": "bad_verification_code"
}
```

You must generate a **new authorization code** each time you test the OAuth Authorization Code flow.

Step 5: Exchange Authorization Code for Access Token

Send POST request

```
POST
```

```
https://github.com/login/oauth/access_token
```

Headers

```
Accept: application/json
```

Body

```
client_id

client_secret

code
```

Response

```
{
  "access_token": "gho_XXXXXXXXXXXXXXXXXXXX",
  "token_type": "bearer",
  "scope": "repo,user"
}
```

Now you have your OAuth2 token.

Step 6: Create Playwright Project

```
npm init playwright@latest
```

Install dependencies

```
npm install
```

Step 7: Create Environment File

```
.env
```

```
GITHUB_TOKEN=gho_XXXXXXXXXXXXXXXXXXXX
```

Install dotenv

```
npm install dotenv
```

Step 8: Load Environment Variables

```
import dotenv from 'dotenv';

dotenv.config();
```

Step 9: First GitHub API Request

Endpoint

```
GET /user
```

Playwright

```
import { test, expect } from '@playwright/test';
import dotenv from 'dotenv';

dotenv.config();

test('Get Logged In User', async ({ request }) => {

  const response = await request.get(
    'https://api.github.com/user',
    {
      headers: {
        Authorization: `Bearer ${process.env.GITHUB_TOKEN}`,
        Accept: 'application/vnd.github+json'
      }
    }
  );

  expect(response.status()).toBe(200);

  const body = await response.json();

  console.log(body);
});
```

Response

```
{
  "login": "john",
  "id": 123456,
  "name": "John Smith",
  "followers": 20
}
```

Assertions

```
expect(body.login).toBeTruthy();
expect(body.id).toBeGreaterThan(0);
expect(body.name).not.toBeNull();
```

Step 10: Get Your Repositories

GET

/user/repos

```
const response = await request.get(
  'https://api.github.com/user/repos',
  {
    headers: {
      Authorization: `Bearer ${process.env.GITHUB_TOKEN}`
    }
  });

expect(response.ok()).toBeTruthy();
```

Step 11: Create Repository

POST

/user/repos

Body

```
{
  "name": "PlaywrightDemoRepo",
  "private": false
}
```

Playwright

```
const response = await request.post(
  'https://api.github.com/user/repos',
  {
    headers: {
      Authorization: `Bearer ${process.env.GITHUB_TOKEN}`
    },
    data: {
      name: 'PlaywrightDemoRepo',
      private: false
    }
  });
```

Expected

201 Created

Step 12: Verify Repository

GET

/repos/{username}/PlaywrightDemoRepo

Verify

```
expect (body.name) .toBe ("PlaywrightDemoRepo");
```

Step 13: Delete Repository

DELETE

```
/repos/{owner}/{repo}
```

Expected

204 No Content

Step 14: API Chaining

Create Repository

↓

Get Repository

↓

Update Repository

↓

Delete Repository

This is one of the most common interview questions.

Step 15: Negative Testing

Invalid Token

Bearer ABC123

Expected

401 Unauthorized

No Token

Expected

401 Unauthorized

Invalid Repository

404 Not Found

Duplicate Repository

422 Unprocessable Entity

Common GitHub REST APIs

API	Method
Get User	GET
Get Repositories	GET
Create Repository	POST
Update Repository	PATCH
Delete Repository	DELETE
List Branches	GET
Create Issue	POST
Close Issue	PATCH
List Pull Requests	GET
Get Commits	GET

Recommended Project Structure

```

GitHubAPIProject

  tests
    github.spec.ts

  utils
    auth.ts

  playwright.config.ts

  .env

  package.json

```

Best Practices

- Never hardcode the access token in your source code.
- Store secrets in a `.env` file and exclude it from version control.

- Reuse authentication headers through a helper or fixture to avoid duplication.
 - Validate both HTTP status codes and response payloads.
 - Clean up test data (such as repositories) after each test to keep your account tidy.
 - Use unique repository names (for example, append a timestamp) to avoid `422` duplicate-name errors.
-